

C++

TAREA-HASH-CSV

cpp-hash

estudiantes.csv

main.cpp

main.exe

evidencias

java-hashmap

estudiantes.csv

cpp-hash

main.cpp

Estudiante

skill_score

mostrata

Aa ab * ? of 2

↑ ↓ ≡ ×

```

1  #include <iostream>
2  #include <fstream>
3  #include <sstream>
4  #include <string>
5
6  using namespace std;
7
8
9  //Julio Cesar Xam 9941-24-8897
10
11 struct Estudiante {
12     int student_id;
13     string full_name;
14     string career;
15     int semester;
16     float gpa;
17     int skill_score;
18 };
19
20
21 struct Nodo {
22     Estudiante estudiante;
23     Nodo* siguiente;
24 };
25
26
27 class HashTable {
28 private:
29     static const int SIZE = 20;
30     Nodo* tabla[SIZE];
31
32     int totalElementos;
33     int totalColisiones;
34
35     int funcionHash(int id) {
36         return id % SIZE;
37     }
38 public:
39     HashTable() {
40         for (int i = 0; i < SIZE; i++) {
41             tabla[i] = nullptr;
42         }
43     }
44
45     void cargarCSV() {
46         ifstream file("estudiantes.csv");
47         if (!file.is_open()) {
48             cout << "Error al abrir el archivo estudiantes.csv" << endl;
49             return;
50         }
51         string line;
52         while (getline(file, line)) {
53             stringstream ss(line);
54             string id, name, career, semester, gpa, skill;
55             ss >> id >> name >> career >> semester >> gpa >> skill;
56             Estudiante estudiante;
57             estudiante.student_id = stoi(id);
58             estudiante.full_name = name;
59             estudiante.career = career;
60             estudiante.semester = stoi(semester);
61             estudiante.gpa = stof(gpa);
62             estudiante.skill_score = stoi(skill);
63             agregar(estudiante);
64         }
65         file.close();
66     }
67
68     void buscarEstudiante(int id) {
69         int index = funcionHash(id);
70         if (tabla[index] != nullptr) {
71             cout << "Buscando estudiante con ID: " << id << endl;
72             while (true) {
73                 Estudiante estudiante = tabla[index]->estudiante;
74                 if (estudiante.student_id == id) {
75                     mostrarEstudiante(estudiante);
76                     return;
77                 }
78                 if (estudiante.siguiente == nullptr) break;
79                 index++;
80                 if (index == SIZE) index = 0;
81             }
82         }
83         cout << "Estudiante no encontrado." << endl;
84     }
85
86     void eliminarEstudiante(int id) {
87         int index = funcionHash(id);
88         if (tabla[index] != nullptr) {
89             cout << "Eliminando estudiante con ID: " << id << endl;
90             while (true) {
91                 Estudiante estudiante = tabla[index]->estudiante;
92                 if (estudiante.student_id == id) {
93                     eliminarNodo(index);
94                     return;
95                 }
96                 if (estudiante.siguiente == nullptr) break;
97                 index++;
98                 if (index == SIZE) index = 0;
99             }
100        }
101        cout << "Estudiante no encontrado." << endl;
102    }
103
104     void mostrarTablaHash() {
105         cout << "Mostrando tabla hash:" << endl;
106         for (int i = 0; i < SIZE; i++) {
107             if (tabla[i] != nullptr) {
108                 Estudiante estudiante = tabla[i]->estudiante;
109                 cout << i << " -> (" << estudiante.student_id << ", " << estudiante.full_name << ", " << estudiante.career << ", " << estudiante.semester << ", " << estudiante.gpa << ", " << estudiante.skill_score << " -> NULL" << endl;
110             } else {
111                 cout << i << " -> vacio -> NULL" << endl;
112             }
113         }
114     }
115
116     void mostrarEstadisticas() {
117         cout << "Mostrando estadisticas:" << endl;
118         cout << "Total elementos: " << totalElementos << endl;
119         cout << "Total colisiones: " << totalColisiones << endl;
120     }
121
122     void salir() {
123         cout << "Programa terminado." << endl;
124     }
125
126     void agregar(Estudiante estudiante) {
127         int index = funcionHash(estudiante.student_id);
128         if (tabla[index] == nullptr) {
129             tabla[index] = new Nodo(estudiante);
130             totalElementos++;
131         } else {
132             totalColisiones++;
133             agregar(estudiante);
134         }
135     }
136
137     void eliminarNodo(int index) {
138         if (tabla[index] != nullptr) {
139             Estudiante estudiante = tabla[index]->estudiante;
140             delete tabla[index];
141             tabla[index] = nullptr;
142             totalElementos--;
143         }
144     }
145
146     void mostrarEstudiante(Estudiante estudiante) {
147         cout << "ID: " << estudiante.student_id << endl;
148         cout << "Nombre: " << estudiante.full_name << endl;
149         cout << "Carrera: " << estudiante.career << endl;
150         cout << "Semestre: " << estudiante.semester << endl;
151         cout << "GPA: " << estudiante.gpa << endl;
152         cout << "Skill Score: " << estudiante.skill_score << endl;
153     }
154
155     ~HashTable() {
156         for (int i = 0; i < SIZE; i++) {
157             if (tabla[i] != nullptr) {
158                 delete tabla[i];
159             }
160         }
161     }
162
163     void menu() {
164         int opcion;
165         do {
166             cout << "===== MENU =====" << endl;
167             cout << "1. Cargar CSV" << endl;
168             cout << "2. Buscar estudiante" << endl;
169             cout << "3. Eliminar estudiante" << endl;
170             cout << "4. Mostrar tabla hash" << endl;
171             cout << "5. Mostrar estadisticas" << endl;
172             cout << "6. Salir" << endl;
173             cout << "Opcion: ";
174             opcion = 0;
175             while (opcion < 0 || opcion > 6) {
176                 opcion = 0;
177             }
178             switch (opcion) {
179                 case 1: cargarCSV(); break;
180                 case 2: buscarEstudiante(opcion); break;
181                 case 3: eliminarEstudiante(opcion); break;
182                 case 4: mostrarTablaHash(); break;
183                 case 5: mostrarEstadisticas(); break;
184                 case 6: salir(); break;
185             }
186         } while (opcion != 6);
187     }
188
189     void run() {
190         menu();
191     }
192
193     void runCSV() {
194         cargarCSV();
195     }
196
197     void runBuscar() {
198         buscarEstudiante(1001);
199     }
200
201     void runEliminar() {
202         eliminarEstudiante(1001);
203     }
204
205     void runMostrarTabla() {
206         mostrarTablaHash();
207     }
208
209     void runMostrarEstadisticas() {
210         mostrarEstadisticas();
211     }
212
213     void runSalir() {
214         salir();
215     }
216
217     void runMenu() {
218         menu();
219     }
220
221     void runAgregar() {
222         agregar(estudiante);
223     }
224
225     void runEliminarNodo() {
226         eliminarNodo(index);
227     }
228
229     void runMostrarEstudiante() {
230         mostrarEstudiante(estudiante);
231     }
232
233     void runDestructor() {
234         ~HashTable();
235     }
236
237     void runMenu2() {
238         menu();
239     }
240
241     void runAgregar2() {
242         agregar(estudiante);
243     }
244
245     void runEliminarNodo2() {
246         eliminarNodo(index);
247     }
248
249     void runMostrarEstudiante2() {
250         mostrarEstudiante(estudiante);
251     }
252
253     void runDestructor2() {
254         ~HashTable();
255     }
256
257     void runMenu3() {
258         menu();
259     }
260
261     void runAgregar3() {
262         agregar(estudiante);
263     }
264
265     void runEliminarNodo3() {
266         eliminarNodo(index);
267     }
268
269     void runMostrarEstudiante3() {
270         mostrarEstudiante(estudiante);
271     }
272
273     void runDestructor3() {
274         ~HashTable();
275     }
276
277     void runMenu4() {
278         menu();
279     }
280
281     void runAgregar4() {
282         agregar(estudiante);
283     }
284
285     void runEliminarNodo4() {
286         eliminarNodo(index);
287     }
288
289     void runMostrarEstudiante4() {
290         mostrarEstudiante(estudiante);
291     }
292
293     void runDestructor4() {
294         ~HashTable();
295     }
296
297     void runMenu5() {
298         menu();
299     }
300
301     void runAgregar5() {
302         agregar(estudiante);
303     }
304
305     void runEliminarNodo5() {
306         eliminarNodo(index);
307     }
308
309     void runMostrarEstudiante5() {
310         mostrarEstudiante(estudiante);
311     }
312
313     void runDestructor5() {
314         ~HashTable();
315     }
316
317     void runMenu6() {
318         menu();
319     }
320
321     void runAgregar6() {
322         agregar(estudiante);
323     }
324
325     void runEliminarNodo6() {
326         eliminarNodo(index);
327     }
328
329     void runMostrarEstudiante6() {
330         mostrarEstudiante(estudiante);
331     }
332
333     void runDestructor6() {
334         ~HashTable();
335     }
336
337     void runMenu7() {
338         menu();
339     }
340
341     void runAgregar7() {
342         agregar(estudiante);
343     }
344
345     void runEliminarNodo7() {
346         eliminarNodo(index);
347     }
348
349     void runMostrarEstudiante7() {
350         mostrarEstudiante(estudiante);
351     }
352
353     void runDestructor7() {
354         ~HashTable();
355     }
356
357     void runMenu8() {
358         menu();
359     }
360
361     void runAgregar8() {
362         agregar(estudiante);
363     }
364
365     void runEliminarNodo8() {
366         eliminarNodo(index);
367     }
368
369     void runMostrarEstudiante8() {
370         mostrarEstudiante(estudiante);
371     }
372
373     void runDestructor8() {
374         ~HashTable();
375     }
376
377     void runMenu9() {
378         menu();
379     }
380
381     void runAgregar9() {
382         agregar(estudiante);
383     }
384
385     void runEliminarNodo9() {
386         eliminarNodo(index);
387     }
388
389     void runMostrarEstudiante9() {
390         mostrarEstudiante(estudiante);
391     }
392
393     void runDestructor9() {
394         ~HashTable();
395     }
396
397     void runMenu10() {
398         menu();
399     }
400
401     void runAgregar10() {
402         agregar(estudiante);
403     }
404
405     void runEliminarNodo10() {
406         eliminarNodo(index);
407     }
408
409     void runMostrarEstudiante10() {
410         mostrarEstudiante(estudiante);
411     }
412
413     void runDestructor10() {
414         ~HashTable();
415     }
416
417     void runMenu11() {
418         menu();
419     }
420
421     void runAgregar11() {
422         agregar(estudiante);
423     }
424
425     void runEliminarNodo11() {
426         eliminarNodo(index);
427     }
428
429     void runMostrarEstudiante11() {
430         mostrarEstudiante(estudiante);
431     }
432
433     void runDestructor11() {
434         ~HashTable();
435     }
436
437     void runMenu12() {
438         menu();
439     }
440
441     void runAgregar12() {
442         agregar(estudiante);
443     }
444
445     void runEliminarNodo12() {
446         eliminarNodo(index);
447     }
448
449     void runMostrarEstudiante12() {
450         mostrarEstudiante(estudiante);
451     }
452
453     void runDestructor12() {
454         ~HashTable();
455     }
456
457     void runMenu13() {
458         menu();
459     }
460
461     void runAgregar13() {
462         agregar(estudiante);
463     }
464
465     void runEliminarNodo13() {
466         eliminarNodo(index);
467     }
468
469     void runMostrarEstudiante13() {
470         mostrarEstudiante(estudiante);
471     }
472
473     void runDestructor13() {
474         ~HashTable();
475     }
476
477     void runMenu14() {
478         menu();
479     }
480
481     void runAgregar14() {
482         agregar(estudiante);
483     }
484
485     void runEliminarNodo14() {
486         eliminarNodo(index);
487     }
488
489     void runMostrarEstudiante14() {
490         mostrarEstudiante(estudiante);
491     }
492
493     void runDestructor14() {
494         ~HashTable();
495     }
496
497     void runMenu15() {
498         menu();
499     }
500
501     void runAgregar15() {
502         agregar(estudiante);
503     }
504
505     void runEliminarNodo15() {
506         eliminarNodo(index);
507     }
508
509     void runMostrarEstudiante15() {
510         mostrarEstudiante(estudiante);
511     }
512
513     void runDestructor15() {
514         ~HashTable();
515     }
516
517     void runMenu16() {
518         menu();
519     }
520
521     void runAgregar16() {
522         agregar(estudiante);
523     }
524
525     void runEliminarNodo16() {
526         eliminarNodo(index);
527     }
528
529     void runMostrarEstudiante16() {
530         mostrarEstudiante(estudiante);
531     }
532
533     void runDestructor16() {
534         ~HashTable();
535     }
536
537     void runMenu17() {
538         menu();
539     }
540
541     void runAgregar17() {
542         agregar(estudiante);
543     }
544
545     void runEliminarNodo17() {
546         eliminarNodo(index);
547     }
548
549     void runMostrarEstudiante17() {
550         mostrarEstudiante(estudiante);
551     }
552
553     void runDestructor17() {
554         ~HashTable();
555     }
556
557     void runMenu18() {
558         menu();
559     }
560
561     void runAgregar18() {
562         agregar(estudiante);
563     }
564
565     void runEliminarNodo18() {
566         eliminarNodo(index);
567     }
568
569     void runMostrarEstudiante18() {
570         mostrarEstudiante(estudiante);
571     }
572
573     void runDestructor18() {
574         ~HashTable();
575     }
576
577     void runMenu19() {
578         menu();
579     }
580
581     void runAgregar19() {
582         agregar(estudiante);
583     }
584
585     void runEliminarNodo19() {
586         eliminarNodo(index);
587     }
588
589     void runMostrarEstudiante19() {
590         mostrarEstudiante(estudiante);
591     }
592
593     void runDestructor19() {
594         ~HashTable();
595     }
596
597     void runMenu20() {
598         menu();
599     }
600
601     void runAgregar20() {
602         agregar(estudiante);
603     }
604
605     void runEliminarNodo20() {
606         eliminarNodo(index);
607     }
608
609     void runMostrarEstudiante20() {
610         mostrarEstudiante(estudiante);
611     }
612
613     void runDestructor20() {
614         ~HashTable();
615     }
616
617     void runMenu21() {
618         menu();
619     }
620
621     void runAgregar21() {
622         agregar(estudiante);
623     }
624
625     void runEliminarNodo21() {
626         eliminarNodo(index);
627     }
628
629     void runMostrarEstudiante21() {
630         mostrarEstudiante(estudiante);
631     }
632
633     void runDestructor21() {
634         ~HashTable();
635     }
636
637     void runMenu22() {
638         menu();
639     }
640
641     void runAgregar22() {
642         agregar(estudiante);
643     }
644
645     void runEliminarNodo22() {
646         eliminarNodo(index);
647     }
648
649     void runMostrarEstudiante22() {
650         mostrarEstudiante(estudiante);
651     }
652
653     void runDestructor22() {
654         ~HashTable();
655     }
656
657     void runMenu23() {
658         menu();
659     }
660
661     void runAgregar23() {
662         agregar(estudiante);
663     }
664
665     void runEliminarNodo23() {
666         eliminarNodo(index);
667     }
668
669     void runMostrarEstudiante23() {
670         mostrarEstudiante(estudiante);
671     }
672
673     void runDestructor23() {
674         ~HashTable();
675     }
676
677     void runMenu24() {
678         menu();
679     }
680
681     void runAgregar24() {
682         agregar(estudiante);
683     }
684
685     void runEliminarNodo24() {
686         eliminarNodo(index);
687     }
688
689     void runMostrarEstudiante24() {
690         mostrarEstudiante(estudiante);
691     }
692
693     void runDestructor24() {
694         ~HashTable();
695     }
696
697     void runMenu25() {
698         menu();
699     }
700
701     void runAgregar25() {
702         agregar(estudiante);
703     }
704
705     void runEliminarNodo25() {
706         eliminarNodo(index);
707     }
708
709     void runMostrarEstudiante25() {
710         mostrarEstudiante(estudiante);
711     }
712
713     void runDestructor25() {
714         ~HashTable();
715     }
716
717     void runMenu26() {
718         menu();
719     }
720
721     void runAgregar26() {
722         agregar(estudiante);
723     }
724
725     void runEliminarNodo26() {
726         eliminarNodo(index);
727     }
728
729     void runMostrarEstudiante26() {
730         mostrarEstudiante(estudiante);
731     }
732
733     void runDestructor26() {
734         ~HashTable();
735     }
736
737     void runMenu27() {
738         menu();
739     }
740
741     void runAgregar27() {
742         agregar(estudiante);
743     }
744
745     void runEliminarNodo27() {
746         eliminarNodo(index);
747     }
748
749     void runMostrarEstudiante27() {
750         mostrarEstudiante(estudiante);
751     }
752
753     void runDestructor27() {
754         ~HashTable();
755     }
756
757     void runMenu28() {
758         menu();
759     }
760
761     void runAgregar28() {
762         agregar(estudiante);
763     }
764
765     void runEliminarNodo28() {
766         eliminarNodo(index);
767     }
768
769     void runMostrarEstudiante28() {
770         mostrarEstudiante(estudiante);
771     }
772
773     void runDestructor28() {
774         ~HashTable();
775     }
776
777     void runMenu29() {
778         menu();
779     }
780
781     void runAgregar29() {
782         agregar(estudiante);
783     }
784
785     void runEliminarNodo29() {
786         eliminarNodo(index);
787     }
788
789     void runMostrarEstudiante29() {
790         mostrarEstudiante(estudiante);
791     }
792
793     void runDestructor29() {
794         ~HashTable();
795     }
796
797     void runMenu30() {
798         menu();
799     }
800
801     void runAgregar30() {
802         agregar(estudiante);
803     }
804
805     void runEliminarNodo30() {
806         eliminarNodo(index);
807     }
808
809     void runMostrarEstudiante30() {
810         mostrarEstudiante(estudiante);
811     }
812
813     void runDestructor30() {
814         ~HashTable();
815     }
816
817     void runMenu31() {
818         menu();
819     }
820
821     void runAgregar31() {
822         agregar(estudiante);
823     }
824
825     void runEliminarNodo31() {
826         eliminarNodo(index);
827     }
828
829     void runMostrarEstudiante31() {
830         mostrarEstudiante(estudiante);
831     }
832
833     void runDestructor31() {
834         ~HashTable();
835     }
836
837     void runMenu32() {
838         menu();
839     }
840
841     void runAgregar32() {
842         agregar(estudiante);
843     }
844
845     void runEliminarNodo32() {
846         eliminarNodo(index);
847     }
848
849     void runMostrarEstudiante32() {
850         mostrarEstudiante(estudiante);
851     }
852
853     void runDestructor32() {
854         ~HashTable();
855     }
856
857     void runMenu33() {
858         menu();
859     }
860
861     void runAgregar33() {
862         agregar(estudiante);
863     }
864
865     void runEliminarNodo33() {
866         eliminarNodo(index);
867     }
868
869     void runMostrarEstudiante33() {
870         mostrarEstudiante(estudiante);
871     }
872
873     void runDestructor33() {
874         ~HashTable();
875     }
876
877     void runMenu34() {
878         menu();
879     }
880
881     void runAgregar34() {
882         agregar(estudiante);
883     }
884
885     void runEliminarNodo34() {
886         eliminarNodo(index);
887     }
888
889     void runMostrarEstudiante34() {
890         mostrarEstudiante(estudiante);
891     }
892
893     void runDestructor34() {
894         ~HashTable();
895     }
896
897     void runMenu35() {
898         menu();
899     }
900
901     void runAgregar35() {
902         agregar(estudiante);
903     }
904
905     void runEliminarNodo35() {
906         eliminarNodo(index);
907     }
908
909     void runMostrarEstudiante35() {
910         mostrarEstudiante(estudiante);
911     }
912
913     void runDestructor35() {
914         ~HashTable();
915     }
916
917     void runMenu36() {
918         menu();
919     }
920
921     void runAgregar36() {
922         agregar(estudiante);
923     }
924
925     void runEliminarNodo36() {
926         eliminarNodo(index);
927     }
928
929     void runMostrarEstudiante36() {
930         mostrarEstudiante(estudiante);
931     }
932
933     void runDestructor36() {
934         ~HashTable();
935     }
936
937     void runMenu37() {
938         menu();
939     }
940
941     void runAgregar37() {
942         agregar(estudiante);
943     }
944
945     void runEliminarNodo37() {
946         eliminarNodo(index);
947     }
948
949     void runMostrarEstudiante37() {
950         mostrarEstudiante(estudiante);
951     }
952
953     void runDestructor37() {
954         ~HashTable();
955     }
956
957     void runMenu38() {
958         menu();
959     }
960
961     void runAgregar38() {
962         agregar(estudiante);
963     }
964
965     void runEliminarNodo38() {
966         eliminarNodo(index);
967     }
968
969     void runMostrarEstudiante38() {
970         mostrarEstudiante(estudiante);
971     }
972
973     void runDestructor38() {
974         ~HashTable();
975     }
976
977     void runMenu39() {
978         menu();
979     }
980
981     void runAgregar39() {
982         agregar(estudiante);
983     }
984
985     void runEliminarNodo39() {
986         eliminarNodo(index);
987     }
988
989     void runMostrarEstudiante39() {
990         mostrarEstudiante(estudiante);
991     }
992
993     void runDestructor39() {
994         ~HashTable();
995     }
996
997     void runMenu40() {
998         menu();
999     }
1000
1001     void runAgregar40() {
1002         agregar(estudiante);
1003     }
1004
1005     void runEliminarNodo40() {
1006         eliminarNodo(index);
1007     }
1008
1009     void runMostrarEstudiante40() {
1010         mostrarEstudiante(estudiante);
1011     }
1012
1013     void runDestructor40() {
1014         ~HashTable();
1015     }
1016
1017     void runMenu41() {
1018         menu();
1019     }
1020
1021     void runAgregar41() {
1022         agregar(estudiante);
1023     }
1024
1025     void runEliminarNodo41() {
1026         eliminarNodo(index);
1027     }
1028
1029     void runMostrarEstudiante41() {
1030         mostrarEstudiante(estudiante);
1031     }
1032
1033     void runDestructor41() {
1034         ~HashTable();
1035     }
1036
1037     void runMenu42() {
1038         menu();
1039     }
1040
1041     void runAgregar42() {
1042         agregar(estudiante);
1043     }
1044
1045     void runEliminarNodo42() {
1046         eliminarNodo(index);
1047     }
1048
1049     void runMostrarEstudiante42() {
1050         mostrarEstudiante(estudiante);
1051     }
1052
1053     void runDestructor42() {
1054         ~HashTable();
1055     }
1056
1057     void runMenu43() {
1058         menu();
1059     }
1060
1061     void runAgregar43() {
1062         agregar(estudiante);
1063     }
1064
1065     void runEliminarNodo43() {
1066         eliminarNodo(index);
1067     }
1068
1069     void runMostrarEstudiante43() {
1070         mostrarEstudiante(estudiante);
1071     }
1072
1073     void runDestructor43() {
1074         ~HashTable();
1075     }
1076
1077     void runMenu44() {
1078         menu();
1079     }
1080
1081     void runAgregar44() {
1082         agregar(estudiante);
1083     }
1084
1085     void runEliminarNodo44() {
1086         eliminarNodo(index);
1087     }
1088
1089     void runMostrarEstudiante44() {
1090         mostrarEstudiante(estudiante);
1091     }
1092
1093     void runDestructor44() {
1094         ~HashTable();
1095     }
1096
1097     void runMenu45() {
1098         menu();
1099     }
1100
1101     void runAgregar45() {
1102         agregar(estudiante);
1103     }
1104
1105     void runEliminarNodo45() {
1106         eliminarNodo(index);
1107     }
1108
1109     void runMostrarEstudiante45() {
1110         mostrarEstudiante(estudiante);
1111     }
1112
1113     void runDestructor45() {
1114         ~HashTable();
1115     }
1116
1117     void runMenu46() {
1118         menu();
1119     }
1120
1121     void runAgregar46() {
1122         agregar(estudiante);
1123     }
1124
1125     void runEliminarNodo46() {
1126         eliminarNodo(index);
1127     }
1128
1129     void runMostrarEstudiante46() {
1130         mostrarEstudiante(estudiante);
1131     }
1132
1133     void runDestructor46() {
1134         ~HashTable();
1135     }
1136
1137     void runMenu47() {
1138         menu();
1139     }
1140
1141     void runAgregar47() {
1142         agregar(estudiante);
1143     }
1144
1145     void runEliminarNodo47() {
1146         eliminarNodo(index);
1147     }
1148
1149     void runMostrarEstudiante47() {
1150         mostrarEstudiante(estudiante);
1151     }
1152
1153     void runDestructor47() {
1154         ~HashTable();
1155     }
1156
1157     void runMenu48() {
1158         menu();
1159     }
1160
1161     void runAgregar48() {
1162         agregar(estudiante);
1163     }
1164
1165     void runEliminarNodo48() {
1166         eliminarNodo(index);
1167     }
1168
1169     void runMostrarEstudiante48() {
1170         mostrarEstudiante(estudiante);
1171     }
1172
1173     void runDestructor48() {
1174         ~HashTable();
1175     }
1176
1177     void runMenu49() {
1178         menu();
1179     }
1180
1181     void runAgregar49() {
1182         agregar(estudiante);
1183     }
1184
1185     void runEliminarNodo49() {
1186         eliminarNodo(index);
1187     }
1188
1189     void runMostrarEstudiante49() {
1190         mostrarEstudiante(estudiante);
1191     }
1192
1193     void runDestructor49() {
1194         ~HashTable();
1195     }
1196
1197     void runMenu50() {
1198         menu();
1199     }
1200
1201     void runAgregar50() {
1202         agregar(estudiante);
1203     }
1204
1205     void runEliminarNodo50() {
1206         eliminarNodo(index);
1207     }
1208
1209     void runMostrarEstudiante50() {
1210         mostrarEstudiante(estudiante);
1211     }
1212
1213     void runDestructor50() {
1214         ~HashTable();
1215     }
1216
1217     void runMenu51() {
1218         menu();
1219     }
1220
1221     void runAgregar51() {
1222         agregar(estudiante);
1223     }
1224
1225     void runEliminarNodo51() {
1226         eliminarNodo(index);
1227     }
1228
1229     void runMostrarEstudiante51() {
1230         mostrarEstudiante(estudiante);
1231     }
1232
1233     void runDestructor51() {
1234         ~HashTable();
1235     }
1236
1237     void runMenu52() {
1238         menu();
1239     }
1240
1241     void runAgregar52() {
1242         agregar(estudiante);
1243     }
1244
1245     void runEliminarNodo52() {
1246         eliminarNodo(index);
1247     }
1248
1249     void runMostrarEstudiante52() {
1250         mostrarEstudiante(estudiante);
1251     }
1252
1253     void runDestructor52() {
1254         ~HashTable();
1255     }
1256
1257     void runMenu53() {
1258         menu();
1259     }
1260
1261     void runAgregar53() {
1262         agregar(estudiante);
1263     }
1264
1265     void runEliminarNodo53() {
1266         eliminarNodo(index);
1267     }
1268
1269     void runMostrarEstudiante53() {
1270         mostrarEstudiante(estudiante);
1271     }
1272
1273     void runDestructor53() {
1274         ~HashTable();
1275     }
1276
1277     void runMenu54() {
1278         menu();
1279     }
1280
1281     void runAgregar54() {
1282         agregar(estudiante);
1283     }
1284
1285     void runEliminarNodo54() {
1286         eliminarNodo(index);
1287     }
1288
1289     void runMostrarEstudiante54() {
1290         mostrarEstudiante(estudiante);
1291     }
1292
1293     void runDestructor54() {
1294         ~HashTable();
1295     }
1296
1297     void runMenu55() {
1298         menu();
1299     }
1300
1301     void runAgregar55() {
1302         agregar(estudiante);
1303     }
1304
1305     void runEliminarNodo55() {
1306         eliminarNodo(index);
1307     }
1308
1309     void runMostrarEstudiante55() {
1310         mostrarEstudiante(estudiante);
1311     }
1312
1313     void runDestructor55() {
1314         ~HashTable();
1315     }
1316
1317     void runMenu56() {
1318         menu();
1319     }
1320
1321     void runAgregar56() {
1322         agregar(estudiante);
1323     }
1324
1325     void runEliminarNodo56() {
1326         eliminarNodo(index);
1327     }
1328
1329     void runMostrarEstudiante56() {
1330         mostrarEstudiante(estudiante);
1331     }
1332
1333     void runDestructor56() {
1334         ~HashTable();
1335     }
1336
1337     void runMenu57() {
1338         menu();
1339     }
1340
1341     void runAgregar57() {
1342         agregar(estudiante);
1343     }
1344
1345     void runEliminarNodo57() {
1346         eliminarNodo(index);
1347     }
1348
1349     void runMostrarEstudiante57() {
1350         mostrarEstudiante(estudiante);
1351     }
1352
1353     void runDestructor57() {
1354         ~HashTable();
1355     }
1356
1357     void runMenu58() {
1358         menu();
1359     }
1360
1361     void runAgregar58() {
1362         agregar(estudiante);
1363     }
1364
1365     void runEliminarNodo58() {
1366         eliminarNodo(index);
1367     }
1368
1369     void runMostrarEstudiante58() {
1370         mostrarEstudiante(estudiante);
1371     }
1372
1373     void runDestructor58() {
1374         ~HashTable();
1375     }
1376
1377     void runMenu59() {
1378         menu();

```

TAREA-HASH-CSV

cpp-hash

estudiantes.csv

main.cpp

main.exe

evidencias

Evidencia 1.png

Evidencia 2.png

java-hashmap

estudiantes.csv

cpp-hash

main.cpp

HashTable

totalColisiones

```
1 #include <iostream>
2 #include <fstream>
3 #include <sstream>
4 #include <string>
5
6 using namespace std;
7
8
9 //Julio Cesar Xam 9941-24-8897
10
11 struct Estudiante {
12     int student_id;
13     string full_name;
14     string career;
15     int semester;
16     float gpa;
17     int skill_score;
18 };
19
20 struct Nodo {
21     Estudiante estudiante;
22     Nodo* siguiente;
23 };
24
25
26 class HashTable {
27
28 private:
29     static const int SIZE = 20;
30     Nodo* tabla[SIZE];
31     int totalElementos;
32     int totalColisiones;
33
34     int funcionHash(int id) {
35         return id % SIZE;
36     }
37
38 public:
39     HashTable() {
40         for (int i = 0; i < SIZE; i++) {
41             tabla[i] = nullptr;
42         }
43     }
44 }
```

mostrarta

Aa ab * 7 of 2

↑ ↓ ≡ ×

===== MENU =====

1. Cargar CSV

2. Buscar estudiante

3. Eliminar estudiante

4. Mostrar tabla hash

5. Mostrar estadísticas

6. Salir

Opcion: 4

===== TABLA HASH =====

Bucket [0] -> vacio -> NULL

Bucket [1] -> (1001, Juan Perez) -> NULL

Bucket [2] -> vacio -> NULL

Bucket [3] -> (1003, Carlos Ruiz) -> NULL

Bucket [4] -> (1004, Maria Gomez) -> NULL

Bucket [5] -> vacio -> NULL

Bucket [6] -> vacio -> NULL

Bucket [7] -> vacio -> NULL

Bucket [8] -> vacio -> NULL

Bucket [9] -> vacio -> NULL

Bucket [10] -> vacio -> NULL

Bucket [11] -> vacio -> NULL

Bucket [12] -> vacio -> NULL

Bucket [13] -> vacio -> NULL

Bucket [14] -> vacio -> NULL

Bucket [15] -> vacio -> NULL

Bucket [16] -> vacio -> NULL

Bucket [17] -> vacio -> NULL

Bucket [18] -> vacio -> NULL

Bucket [19] -> vacio -> NULL

===== MENU =====

1. Cargar CSV

2. Buscar estudiante

3. Eliminar estudiante

4. Mostrar tabla hash

5. Mostrar estadísticas

6. Salir

Opcion: 5

===== ESTADISTICAS =====

Total estudiantes: 3

Total colisiones: 0

Factor de carga: 0.15

===== MENU =====

1. Cargar CSV

2. Buscar estudiante

3. Eliminar estudiante

4. Mostrar tabla hash

5. Mostrar estadísticas

6. Salir

Opcion: 6

Saliendo del programa...

c:\Users\Xamju\OneDrive\Documentos\Universidad\STO SE

OUTLINE

TIMELINE

Java

TAREA-HASH-CSV

cpp-hash

estudiantes.csv

main.cpp

main.exe

evidencias

Evidencia 1.png

Evidencia 2.png

Evidencia 3.png

java-hashmap

Estudiante.class

Estudiante.java

estudiantes.csv

Main.class

Main.java

4

OUTLINE

TIMELINE

PROJECTS

RUN CONFIGURATION

JAVA PROJECTS

java-hashmap > J Main.java > Language Support for Java(TM) by Red Hat > Main > cargarCSV()

```
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.util.HashMap;
4 import java.util.Scanner;
5
6 public class Main {
7
8     static HashMap<Integer, Estudiante> estudiantes =
9         new HashMap<>();
10
11     static Scanner sc = new Scanner(System.in);
12
13     public static void cargarCSV() {
14
15         try {
16
17             BufferedReader br =
18                 new BufferedReader(
19                     new FileReader(fileName: "estudiantes.csv")
20                 );
21
22             String linea;
23
24             br.readLine();
25
26             while ((linea = br.readLine()) != null) {
27
28                 String[] datos = linea.split(regex: ",");
29
30                 int id = Integer.parseInt(datos[0]);
31
32                 if (estudiantes.containsKey(id)) {
33
34                     System.out.println(
35                         "Estudiante duplicado: " + id
36                     );
37
38                     continue;
39                 }
40
41                 Estudiante e = new Estudiante(
42                     id,
43                     datos[1],
44                     datos[2],
45                     Integer.parseInt(datos[3]),
46                     Double.parseDouble(datos[4]),
47                     Integer.parseInt(datos[5])
48                 );
49
50                 estudiantes.put(id, e);
51
52                 System.out.println(
```

-Stderr.encoding=UTF-8

Julio Cesar Xam Chivalan

Carnet: 9941-24-8897

===== MENU =====

1. Cargar CSV

2. Buscar estudiante

3. Eliminar estudiante

4. Mostrar estudiantes

5. Mostrar estadísticas

6. Salir

Opcion: 1

Cargando estudiante: 1001 - Juan Perez

Cargando estudiante: 1002 - Ana Lopez

Cargando estudiante: 1003 - Carlos Ruiz

Cargando estudiante: 1004 - Maria Gomez

Carga completada.

===== MENU =====

1. Cargar CSV

2. Buscar estudiante

3. Eliminar estudiante

4. Mostrar estudiantes

5. Mostrar estadísticas

6. Salir

Opcion: 2

Ingrese ID: 1001

===== ESTUDIANTE ENCONTRADO =====

ID: 1001

Nombre: Juan Perez

Carrera: Ingenieria en Sistemas

Semestre: 5

GPA: 85.5

Skill Score: 120

===== MENU =====

1. Cargar CSV

2. Buscar estudiante

3. Eliminar estudiante

4. Mostrar estudiantes

5. Mostrar estadísticas

6. Salir

Opcion: 3

Ingrese ID: 1002

Estudiante eliminado.

===== MENU =====

1. Cargar CSV

2. Buscar estudiante

3. Eliminar estudiante

4. Mostrar estudiantes

5. Mostrar estadísticas

6. Salir

Opcion: 4

TAREA-HASH-CSV

cpp-hash

estudiantes.csv

main.cpp

main.exe

evidencias

Evidencia 1.png

Evidencia 2.png

Evidencia 3.png

java-hashmap

Estudiante.class

Estudiante.java

estudiantes.csv

Main.class

Main.java

4

OUTLINE

TIMELINE

PROJECTS

RUN CONFIGURATION

JAVA PROJECTS

java-hashmap > J Main.java > Language Support for Java(TM) by Red Hat > Main > cargarCSV()

```
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.util.HashMap;
4 import java.util.Scanner;
5
6 public class Main {
7
8     static HashMap<Integer, Estudiante> estudiantes =
9         new HashMap<>();
10
11     static Scanner sc = new Scanner(System.in);
12
13     public static void cargarCSV() {
14
15         try {
16
17             BufferedReader br =
18                 new BufferedReader(
19                     new FileReader(fileName: "estudiantes.csv")
20                 );
21
22             String linea;
23
24             br.readLine();
25
26             while ((linea = br.readLine()) != null) {
27
28                 String[] datos = linea.split(regex: ",");
29
30                 int id = Integer.parseInt(datos[0]);
31
32                 if (estudiantes.containsKey(id)) {
33
34                     System.out.println(
35                         "Estudiante duplicado: " + id
36                     );
37
38                     continue;
39                 }
40
41                 Estudiante e = new Estudiante(
42                     id,
43                     datos[1],
44                     datos[2],
45                     Integer.parseInt(datos[3]),
46                     Double.parseDouble(datos[4]),
47                     Integer.parseInt(datos[5])
48                 );
49
50                 estudiantes.put(id, e);
51
52                 System.out.println(
```

===== MENU =====

1. Cargar CSV

2. Buscar estudiante

3. Eliminar estudiante

4. Mostrar estudiantes

5. Mostrar estadísticas

6. Salir

Opcion: 4

===== HASHMAP ESTUDIANTES =====

ID: 1001

Nombre: Juan Perez

Carrera: Ingenieria en Sistemas

Semestre: 5

GPA: 85.5

Skill Score: 120

ID: 1003

Nombre: Carlos Ruiz

Carrera: Ingenieria Civil

Semestre: 3

GPA: 78.3

Skill Score: 95

ID: 1004

Nombre: Maria Gomez

Carrera: Ingenieria Electronica

Semestre: 8

GPA: 88.9

Skill Score: 180

===== MENU =====

1. Cargar CSV

2. Buscar estudiante

3. Eliminar estudiante

4. Mostrar estudiantes

5. Mostrar estadísticas

6. Salir

Opcion: 5

===== ESTADISTICAS =====

Total estudiantes: 3

Usando HashMap nativo de Java.

===== MENU =====

1. Cargar CSV

2. Buscar estudiante

3. Eliminar estudiante

4. Mostrar estudiantes

5. Mostrar estadísticas

6. Salir

Opcion: 6

Saliendo...

c:\Users\camju\OneDrive\Documents\Universidad\STO SE
MESTRE\PROGRAMACION 3\tarea-hash-csv\java-hashmap\]